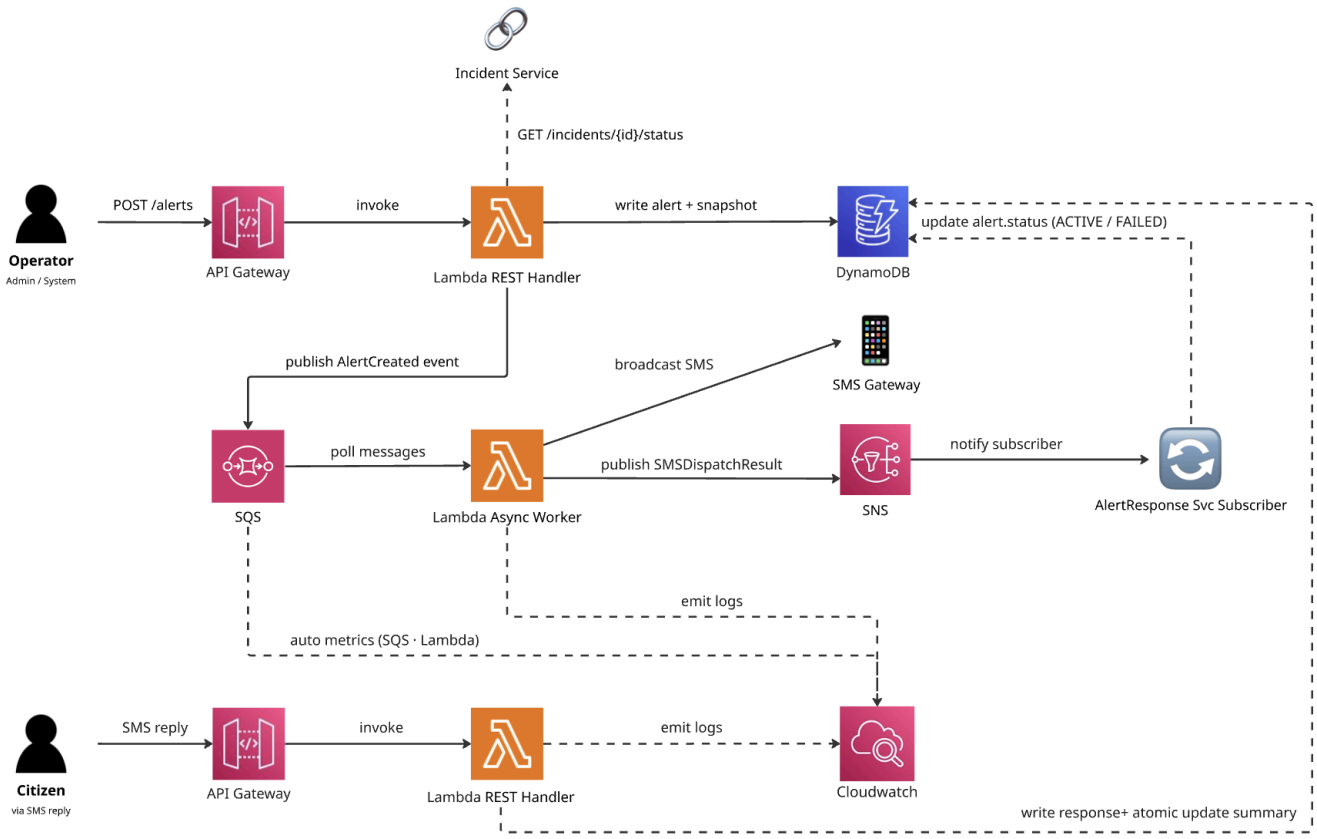
Service Architecture



Components (องค์ประกอบของระบบ)

1. ฝั่งผู้ใช้งาน (Users)

- **Operator (Admin/System):** เจ้าหน้าที่ที่เรียกใช้งาน REST API เพื่อสร้างการแจ้งเตือนภัยและดึงข้อมูลสรุปผล
- **Citizen:** ประชาชนในพื้นที่เสี่ยงภัยที่รับข้อความและส่ง SMS ตอบกลับเพื่อแจ้งสถานะของตนเอง

2. จุดรับคำขอ (API Gateway)

- ทำหน้าที่เป็นประตูหน้าบ้านรับคำขอแบบ Synchronous ทั้งหมด ได้แก่:
  - `POST /alerts` สำหรับรับคำสั่งสร้างการแจ้งเตือน
  - `GET /alerts/{id}` สำหรับดึงข้อมูลสรุปสถิติ
  - `POST /alerts/{id}/responses` สำหรับรับข้อความ SMS ที่ประชาชนตอบกลับ

3. กลุ่มประมวลผลแบบทันที (Synchronous Lambda Handlers)

- **CreateAlertHandler:** รับคำสั่งสร้าง Alert -> วิ่งไปเช็คข้อมูลกับ Service ของเพื่อน -> บันทึกตารางลง DB -> โยนงานเข้าคิว SQS แล้วตอบกลับ `201 Created` ทันทีโดยไม่ต้องรอส่ง SMS เสร็จ
- **GetAlertSummaryHandler:** รับคำขอจาก Operator เพื่อไปดึงยอดสรุปสถิติ (คนรับทราบ/ปลอดภัย/ขอความช่วยเหลือ) จากตาราง `AlertData` มาแสดงผล
- **RecordResponseHandler:** รับข้อความที่ประชาชนตอบกลับมา (เช่น 1, 2, 3) -> ตรวจสอบความถูกต้อง (Validation) -> อัปเดตยอด Counter ใน DB ทันที (Atomic Update) พร้อมเก็บประวัติการตอบกลับ

4. บริการของเพื่อนในชั้นเรียน (Classmate Services - External)

- **IncidentReporter Service:** บริการที่เราวิ่งไปถามแบบ Synchronous เพื่อดึงสถานะเหตุการณ์ (Status), ความรุนแรง (Severity) และประเภทภัยพิบัติ (Incident Type) มาประกอบการส่งข้อความ
- **IncidentImpactZone Service:** บริการที่เราวิ่งไปถามแบบ Synchronous เพื่อขอดึงพิกัดหรือรหัสพื้นที่ (Location) ที่ได้รับผลกระทบ เพื่อเอามากวาดหาเบอร์โทรประชาชน

5. กลุ่มประมวลผลเบื้องหลัง (Asynchronous Messaging & Workers)

- **Amazon SQS (Queue):** คิวรับพักงาน (Event: `AlertCreated`) เพื่อไม่ให้หน้าบ้านค้างระหว่างรอส่งข้อความ
- **SMSDispatchWorker (Lambda):** ดึงงานจาก SQS ไปไล่ส่ง SMS หาประชาชนตามพื้นที่ (มีระบบ Retry หากส่งไม่สำเร็จ) และเมื่อส่งครบจะประกาศผลลัพธ์ออกไปทาง SNS
- **Amazon SNS (Topic):** ช่องทางสำหรับประกาศผลลัพธ์การส่ง SMS (`SMSDispatchResult`)
- **AlertResponseSubscriber (Lambda):** คอยฟังเสียงจาก SNS พอได้ยินว่าส่ง SMS เสร็จแล้ว จะวิ่งไปเปลี่ยนสถานะ Alert ใน DynamoDB เป็น `ACTIVE` หรือ `FAILED`

6. ฐานข้อมูลและระบบมอนิเตอร์ (Storage & Monitoring)

- **DynamoDB:** จัดเก็บข้อมูลหลักของระบบ ได้แก่ `AlertData` (เก็บสถานะและยอดสรุป) และ `ResponseData` (เก็บประวัติการตอบกลับรายคน)
- **CloudWatch:** ระบบหลังบ้านสำหรับเก็บ Logs ของ Lambda ทุกตัว และคอยมอนิเตอร์สุขภาพของระบบ (Metrics)

Explanation (อธิบายการทำงานของระบบ)

สถาปัตยกรรมของ AlertResponse Service ถูกออกแบบมาเพื่อรองรับงานที่ต้องการความรวดเร็ว (Synchronous) และงานที่ต้องใช้เวลาประมวลผลนาน (Asynchronous) อย่างเป็นระบบครับ โดยมีเวิร์กโฟลว์การทำงานดังนี้:

1. การสร้างและกระจายการแจ้งเตือน (Create & Broadcast Flow) เมื่อเจ้าหน้าที่ (Operator) ต้องการประกาศเตือนภัย จะส่งคำสั่งผ่าน API Gateway เข้ามาที่ `CreateAlertHandler` ระบบจะไม่ทำงานแบบโดดเดี่ยว แต่จะวิ่งไปดึงข้อมูลความรุนแรงจาก **IncidentReporter Service** และดึงขอบเขตพื้นที่จาก **IncidentImpactZone Service** (ซึ่งเป็น Service ของเพื่อน) เพื่อความสมบูรณ์ของข้อมูล

จากนั้นระบบจะบันทึกข้อมูลลง DynamoDB โยนคำสั่งส่ง SMS เข้าคิว **Amazon SQS** และตอบกลับ `201 Created` ให้เจ้าหน้าที่ทันทีโดยไม่ปล่อยให้ระบบรอนาน

หลังจากนั้น **SMSDispatchWorker** จะมารับช่วงต่อโดยดึงงานจาก SQS ไปไล่ส่ง SMS แบบเบื้องหลัง (Asynchronous) เมื่อส่งเสร็จสิ้น มันจะประกาศผลลัพธ์ผ่าน **Amazon SNS** ซึ่งจะมี **AlertResponseSubscriber** คอยดักฟังอยู่ เพื่อรับหน้าที่ไปอัปเดตสถานะสุดท้าย (ACTIVE/FAILED) ลงใน DynamoDB การออกแบบแนวนี้ช่วยลดภาระการทำงาน (Decoupling) และทำให้ระบบทนทานขึ้นครับ

2. การโต้ตอบกับประชาชน (Citizen Response Flow) เมื่อประชาชนได้รับ SMS และพิมพ์ตอบกลับ (เช่น 1,2,3) ข้อความจะวิ่งผ่าน API Gateway มาที่ `RecordResponseHandler` ระบบจะทำการตรวจสอบความถูกต้องของข้อมูล (Validation) และทำการอัปเดตยอดสถิติแบบ Atomic Update ลงใน DynamoDB ทันที เพื่อให้ข้อมูลเป็นปัจจุบันที่สุด

3. การติดตามผลลัพธ์ (Monitoring Flow) ในระหว่างที่เหตุการณ์กำลังดำเนินอยู่ เจ้าหน้าที่สามารถเรียก API ไปที่ `GetAlertSummaryHandler` เพื่อดึงยอดสรุป (เช่น มีคนปลอดภัยกี่คน, ต้องการความช่วยเหลือกี่คน) ไปแสดงบน Dashboard ได้ทันที นอกจากนี้ การทำงานของระบบทั้งหมดจะถูกส่งไปบันทึก Logs และ Metrics ที่ **Amazon CloudWatch** เพื่อให้มั่นใจว่าระบบสามารถทำงานได้อย่างเสถียรในสถานการณ์ฉุกเฉินครับ